● Official Technical Specification v1.0

SERRO (Modern Viewport Motion Engine)

Lightweight viewport animation trigger, kinetic typography engine, velocity marquee stream, and progressive editorial scroll scrubber. Engineered for hardware-accelerated fluid motion with zero dependencies.

Author:

License:

Architecture:

Compositing:

1. Overview & Architecture

SERRO is a fast, zero-dependency browser motion system designed to replace heavy legacy animation libraries. Built from the ground up around the modern `IntersectionObserver` API and pure CSS GPU compositing pipelines (`transform` , `opacity` , `filter` , `clip-path`), SERRO achieves rock-solid, fluid motion on both desktop and mobile devices.

Unlike legacy libraries that trigger reflows by calculating element bounding boxes on scroll events, MOS uses a decoupled asynchronous lifecycle:

> **Core Engine Principles**
>
> - **Zero Layout Shifts (CLS 0.0)**: No dimensions, margins, or padding are mutated during animation execution.
> - **Two-Way Scroll Lifecycle**: Elements animate in on entry and reset cleanly when scrolled out of view, replaying on demand.
> - **Anti-Flicker Boundary Hysteresis**: An 80px boundary clearance buffer prevents layout chattering when elements near viewport edges.
> - **Pure CSS Keyframe Engines**: Complex character and word split animations run entirely on browser compositor threads.

2. Quick Start & CDN Setup

Drop the official CDN links into your HTML document. No package managers, build steps, or compilation required:

HTML • In <head>

```
<!-- Official SERRO Stylesheet -->
<link rel="stylesheet" href="https://cdn.jsdelivr.net/gh/soundarrajan-a/serro@main/serro.css">
```

HTML • Before </body>

```
<!-- Official SERRO JavaScript Engine -->
<script src="https://cdn.jsdelivr.net/gh/soundarrajan-a/serro@main/serro.js"></script>
<script>
  SERRO.init();
</script>
```

Minimal Complete Example

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <link rel="stylesheet" href="https://cdn.jsdelivr.net/gh/soundarrajan-a/serro@main/serro.css">
</head>
<body>

  <h1 data-serro="text-slide-up">Hello World</h1>
  <div data-serro="blur-up" data-serro-duration="1200">
    <p>This element animates smoothly when scrolled into view.</p>
  </div>

  <script src="https://cdn.jsdelivr.net/gh/soundarrajan-a/serro@main/serro.js"></script>
  <script>
    SERRO.init();
  </script>
</body>
</html>
```

3. Viewport Scroll Animation Presets (28 Core Presets)

Attach the `data-mos` attribute to any HTML element. Customize transition timing with `data-mos-duration` (in milliseconds or seconds) and `data-mos-delay`:

CATEGORY	ATTRIBUTE VALUE	GPU COMPOSITOR BEHAVIOR	DEFAULT EASING
Lens & Blur	`data-mos="blur-up"`	Translates +30px Y with 8px optical bokeh blur	cubic-bezier(0.16, 1, 0.3, 1)
Lens & Blur	`data-mos="blur-down"`	Translates -30px Y with 8px optical bokeh blur	cubic-bezier(0.16, 1, 0.3, 1)
Lens & Blur	`data-mos="blur-left"`	Translates +35px X with 8px optical bokeh blur	cubic-bezier(0.16, 1, 0.3, 1)
Lens & Blur	`data-mos="blur-right"`	Translates -35px X with 8px optical bokeh blur	cubic-bezier(0.16, 1, 0.3, 1)
Lens & Blur	`data-mos="blur-scale"`	Scales from 0.90 with 8px bokeh diffusion	cubic-bezier(0.16, 1, 0.3, 1)
Lens & Blur	`data-mos="lens-flare"`	Scales from 1.10 with high-contrast brightness flare	cubic-bezier(0.16, 1, 0.3, 1)
3D Spatial	`data-mos="tilt-up"`	18° X-axis perspective pitch rotation + 30px Y	preserve-3d
3D Spatial	`data-mos="tilt-down"`	-18° X-axis perspective pitch rotation - 30px Y	preserve-3d
3D Spatial	`data-mos="tilt-left"`	-16° Y-axis perspective yaw rotation - 30px X	preserve-3d
3D Spatial	`data-mos="tilt-right"`	16° Y-axis perspective yaw rotation + 30px X	preserve-3d
3D Spatial	`data-mos="flip-x"`	70° perspective flip along horizontal baseline	preserve-3d
3D Spatial	`data-mos="flip-y"`	70° perspective flip along vertical baseline	preserve-3d

CATEGORY	ATTRIBUTE VALUE	GPU COMPOSITOR BEHAVIOR	DEFAULT EASING
Shape Wipes	`data-mos="wipe-up"`	Geometric upward clip-path inset reveal	clip-path: inset()
Shape Wipes	`data-mos="wipe-down"`	Geometric downward clip-path inset reveal	clip-path: inset()
Shape Wipes	`data-mos="wipe-left"`	Geometric leftward clip-path inset reveal	clip-path: inset()
Shape Wipes	`data-mos="wipe-right"`	Geometric rightward clip-path inset reveal	clip-path: inset()
Shape Wipes	`data-mos="circle-expand"`	Radial circle(140%) clip-path expansion	clip-path: circle()
Shape Wipes	`data-mos="diagonal-wipe"`	Polygon angular curtain diagonal wipe	clip-path: polygon()
Springs & Fades	`data-mos="fade-up"`	Classic upward translation with fade in	cubic-bezier(0.16, 1, 0.3, 1)
Springs & Fades	`data-mos="fade-down"`	Classic downward translation with fade in	cubic-bezier(0.16, 1, 0.3, 1)
Springs & Fades	`data-mos="spring-pop"`	Scale(0.7) overshoot to scale(1.0) with recoil	cubic-bezier(0.34, 1.56, 0.64, 1)
Springs & Fades	`data-mos="soft-bounce"`	Damped spring settling translation	cubic-bezier(0.34, 1.3, 0.64, 1)

4. Velocity Marquee Ticker Engine

The Velocity Marquee provides continuous, hardware-accelerated horizontal text streaming with automatic infinite looping. It supports multi-element typography, glowing separator dots, outline strokes, and automatic single-row duplication.

Standard Marquee Markup

HTML Markup

```
<div class="mos-ticker-wrap">
  <div class="mos-ticker-track-left">
    <div class="ticker-row">
      <span class="ticker-item ticker-item-solid">ULTRA LIGHTWEIGHT <span class="ticker-dot"></span></span>
      <span class="ticker-item ticker-item-outline">ZERO DEPENDENCIES <span class="ticker-dot"></span></span>
      <span class="ticker-item ticker-item-accent">FLUID VELOCITY MOTION <span class="ticker-dot"></span></span>
    </div>
    <!-- Duplicate row for seamless infinite looping -->
    <div class="ticker-row" aria-hidden="true">
      <span class="ticker-item ticker-item-solid">ULTRA LIGHTWEIGHT <span class="ticker-dot"></span></span>
      <span class="ticker-item ticker-item-outline">ZERO DEPENDENCIES <span class="ticker-dot"></span></span>
      <span class="ticker-item ticker-item-accent">FLUID VELOCITY MOTION <span class="ticker-dot"></span></span>
    </div>
  </div>
</div>
```

Automatic Features in MOS Marquee

1. **Reverse Direction**: Use `class="mos-ticker-track-right"` for reverse motion.
2. **Auto-Duplication**: If only one `.ticker-row` is provided, `mos.js` automatically duplicates it with `aria-hidden="true"` to guarantee an infinite loop.
3. **Hover Pause**: Hovering over the ticker automatically pauses animation smoothly via pure CSS.

5. Editorial Scroll Scrub

The Editorial Scroll Scrub dynamically binds paragraph text to viewport scroll progression. As the user travels down the page, every word progressively illuminates from 18% opacity to high-contrast 100% brilliance, with subtle glow effects and keyword highlighting.

Option A: Full Stage with Progress Meter

HTML • Stage with Meter

```
<div class="scrub-stage">
  <div class="scrub-meter-bar">
    <span>SCROLL PROGRESSION</span>
    <div class="scrub-progress-track">
      <div class="scrub-progress-fill"></div>
    </div>
    <span class="scrub-percent-label">0%</span>
  </div>

  <p class="scrub-paragraph" data-mos-scrub-highlight="motion,viewport,scroll,kinetic">
    Motion that responds directly to your scroll position. Every word dynamically illuminates
    from deep dark to high-contrast brilliance with GPU hardware acceleration.
  </p>
</div>
```

Option B: Standalone Raw Paragraph

You can drop `class="scrub-paragraph"` on any plain `<p>` element without stages or wrappers:

HTML • Raw Markup

```
<p class="scrub-paragraph">
  Pure vanilla paragraph text that tracks its own viewport coordinates automatically.
</p>
```

6. Kinetic Typography & Split-Text Engine

MOS features a built-in kinetic typography engine capable of tokenizing text by word or character and applying staggered kinetic motion presets:

Split Text Attributes

ATTRIBUTE	VALUES	DESCRIPTION
`data-mos-split`	`words` | `chars`	Tokenizes element content into words or individual characters
`data-mos-stagger`	Milliseconds (e.g. `50`, `70`)	Delay gap between consecutive animated spans
`data-mos`	Text Preset Name	Target animation preset applied to each staggered token

Popular Kinetic Presets

HTML Examples

```
<!-- Staggered Words Masked Baseline -->
<h1 data-mos="text-slide-up" data-mos-split="words" data-mos-stagger="70">
  FLUID KINETIC HEADLINE MASKED BASELINE
</h1>

<!-- Staggered Character Elastic Recoil -->
<h2 data-mos="text-stretch-snap" data-mos-split="chars" data-mos-stagger="40">
  MOMENTUM RECOIL
</h2>

<!-- Cyberpunk Scanline Glitch -->
<h2 data-mos="text-glitch" data-mos-duration="1200">
  CYBERNETIC CHANNEL GLITCH
</h2>

<!-- Gas Discharge Neon Glow -->
<h2 data-mos="text-neon-flicker" data-mos-duration="1500">
  HIGH VOLTAGE NEON GLOW
</h2>
```

7. JavaScript API & Configuration Options

MOS exposes a global `window.MOS` object with full programmatic controls:

API Methods

METHOD	PARAMETERS	DESCRIPTION
`MOS.init(options)`	Configuration Object (Optional)	Initializes split-text, marquees, scrubbers, and observers
`MOS.refresh()`	None	Re-scans DOM for dynamically inserted elements / single page apps
`MOS.updateScrubs()`	None	Recalculates scroll progress on all active editorial scrub paragraphs

Configuration Options

```javascript
MOS.init({
  // Trigger once on entry or replay on two-way scroll (default: false)
  once: false,

  // Viewport intersection root margin (default: '0px 0px -40px 0px')
  rootMargin: '0px 0px -40px 0px',

  // Ratio of element visibility required to trigger (default: 0)
  threshold: 0
});
```

8. Anti-Flicker Architecture & Hysteresis Buffer

Legacy scroll libraries often suffer from *Layout Hysteresis*: when an element exits the viewport, removing the animation class restores its initial transform (e.g. `translateY(30px)`), which shifts the element back into the viewport, triggering an infinite rapid chattering flicker.

MOS completely eliminates this issue with a dual-stage safety mechanism:

> The MOS Hysteresis Solution

1. **Sticky Retention at Top of Screen**: Elements that have entered the screen and scrolled past the top boundary *remain rendered*. They are never removed while the user is reading downwards.
2. **80px Clearance Buffer for Reset**: Elements are only reset when they have exited more than 80px below the bottom of the viewport (`rect.top > windowH + 80`). Transform shifts (30–40px) can never bridge this 80px gap, making chattering physically impossible.

9. Browser Compatibility & Benchmarks

BROWSER	MINIMUM VERSION	GPU COMPOSITOR SUPPORT	STATUS
Google Chrome	Version 58+	Full Hardware Acceleration	Supported
Brave Browser	Version 1.0+	Full Hardware Acceleration	Supported
Apple Safari (iOS / macOS)	Version 12.1+	WebKit Hardware Compositing	Supported
Mozilla Firefox	Version 55+	Gecko Quantum Compositor	Supported
Microsoft Edge	Version 79+	Chromium Direct3D Compositor	Supported

10. Frequently Asked Questions (FAQ)

Q: Can I use SERRO without any CSS or JS frameworks?

Yes. SERRO is 100% vanilla HTML, CSS, and JavaScript. It does not require React, Vue, jQuery, or any build steps.

Q: How do I make animations trigger only once?

Pass `once: true` to `SERRO.init({ once: true })`. Elements will animate when first scrolled into view and remain permanently visible.

Q: Why did my ticker wrap vertically in an online sandbox?

If your sandbox cached an older version of `serro.css`, ensure you include `serro.js`. The latest `serro.js` automatically enforces inline flex layout and nowrap rules directly on all ticker rows.

